

ShrimpFlow

程序员的行为建模与协作驱动系统

Record → Visualize → Learn → Become You → Share

每个程序员都是一只虾，ShrimpFlow 让小虾记住自己是怎么长大的，
然后让所有虾都能一夜长成澳洲大龙虾。

Layer 4: AUTOPILOT — Agent 按你的风格自主工作

Layer 3: BRAIN — 从历史中提取模式、习惯、偏好

Layer 2: MIRROR — 美观地展示你的所有操作历史

Layer 1: SHADOW — 记录终端、Git、对话、环境的一切

OmniClaw

March 2026

Contents

1	问题：程序员与 AI 之间缺失的记忆层	2
2	ShrimpFlow 做什么	2
3	四层产品架构	2
3.1	Layer 1: SHADOW — 静默采集	3
3.2	Layer 2: MIRROR — 可视化仪表盘	3
3.3	Layer 3: BRAIN — 行为模型	3
3.4	Layer 4: AUTOPILOT — 数字分身	3
4	ClawProfile — 可编码的开发者行为资产	4
4.1	可组合的行为 Workflow	4
5	ClawProfile 的流通 — 从个人记忆到社区共享	4
5.1	Make Hinton in Your Computer	4
6	技术定位：参数化 vs. 非参数化的行为记忆	5
7	技术架构	5
7.1	统一事件数据模型	6
8	竞品对比	7
9	商业模式	7
10	项目意义	7
10.1	从“用 AI 写代码”到“AI 理解你怎么写代码”	7
10.2	三个层次的价值	7
11	使用场景	8
11.1	具身智能博士生的一天	8
11.2	团队新人的冷启动	8
11.3	开源社区的行为模式市场	8
12	实际效果	8
12.1	Workflow 自主执行验证	8
13	项目定位	9
14	团队	9

问题：程序员与 AI 之间缺失的记忆层

2025 年，AI 编程助手已深度嵌入开发者工作流。一个典型的程序员每天产生数百条终端命令、数十次 Git 操作、若干轮 AI 对话，以及大量隐含在交互中的技术判断和设计决策。但这些数据存在两个结构性问题：

上下文易失。 每轮 AI 对话都是一次性的。关掉窗口，AI 对你的了解归零——它不记得你昨天花两个小时解决的依赖冲突，不记得你上周做的架构选型，更不记得你过去三年积累的技术直觉。每次协作，都是一次冷启动。

行为数据未被利用。 终端历史、Git 日志、AI 对话记录、调试过程——这些数据真实地记录了一个开发者如何思考和工作。但它们散落在不同工具中，以原始日志的形式存在，从未被系统地提取、结构化和利用。

核心洞察。 WakaTime 告诉你“今天写了 3 小时 Python”；ShrimpFlow 告诉你“今天你用 `grep` 定位了一个 race condition，然后用 `git bisect` 找到了引入 bug 的 commit，最后用 `mutex` 修复了它”——然后下次遇到类似问题，你的 Agent 直接按这个模式帮你解决。

本质上，程序员与 AI 之间缺少一个持久的、可进化的行为记忆层。

ShrimpFlow 做什么

ShrimpFlow 在开发者和 AI 之间构建这个缺失的记忆层。核心理念：

Record → Visualize → Learn → Become You → Share

系统采用四层架构，每一层独立提供价值，层层递进最终构建出程序员的数字分身。

四层产品架构



Layer 1: SHADOW — 静默采集

SHADOW 以完全被动、零干扰的方式采集开发过程中的全维度行为数据。所有数据写入本地 SQLite，数据不离开你的机器。内置敏感信息过滤器自动脱敏。

数据源	采集方式	记录内容
终端命令	zsh precmd/preexec hook	每条命令 + 工作目录 + 退出码 + 耗时
Git	post-commit/checkout hook	commit 信息、diff 摘要、分支切换
AI 对话	OpenClaw/Claude hook	你问了什么、Agent 做了什么、用了哪些 skill
IDE	编辑器插件	文件浏览轨迹、编辑热区、重构操作
环境	cron 定时	系统负载、活跃进程、依赖变更

Layer 2: MIRROR — 可视化仪表盘

- 五个精心设计的可视化页面，让你“看见”自己的开发生涯：
- 1. **Timeline**——事件以不同颜色粒子在 Canvas 河流中流淌，密集处形成明亮光带。
 - 2. **Project Galaxy**——每个项目是一颗 3D 星球，大小=活跃度，颜色=语言。
 - 3. **Skill Tree**——RPG 风格径向技能图谱，从操作频率自动计算等级。
 - 4. **Daily Digest**——日历热力图 + AI 第一人称开发日记。
 - 5. **Pattern Radar**——雷达图 + 热力图 + 趋势线，多维能力画像。

Layer 3: BRAIN — 行为模型

BRAIN 从累积数据中提取稳定的个人行为模式，构建 **ClawProfile**。每个模式都有置信度评分（0%-100%），达到 80% 时提醒用户审计入库。

类型	说明	示例
工作流	典型步骤序列	修 bug: git log → grep → 读文件 → 写测试 → 修代码
编码偏好	代码风格习惯	用 const 不用 function; 变量 camelCase
决策策略	选择时的判断倾向	选技术栈看社区活跃度; 遇性能问题先 DevTools
调试策略	排查典型路径	先查日志 → 最小复现 → 二分定位
时间模式	效率规律	高效时段 10:00–12:00; commit 高峰在下午
协作偏好	与 AI 协作习惯	给出详细约束; 偏好分步验证

Layer 4: AUTOPILOT — 数字分身

ClawProfile 直接驱动 AI 协作流程，渐进式提升自动化水平：

级别	模式	行为
Level 1	建议模式	Agent 根据历史行为模式给出建议
Level 2	半自动模式	你下指令，Agent 按你的习惯执行
Level 3	全自动模式	Agent 接管任务队列，按你的方式自主工作

效果：AI 的行为从“通用合理”提升为“对你而言最优”。

ClawProfile — 可编码的开发者行为资产

ClawProfile 是 ShrimpFlow 的核心产物——结构化的、持续更新的开发者行为模型。它不是一次性快照，而是随数据积累**持续进化**的活模型。每个行为模式包含：

- **结构化规则** — 提取出的具体行为模式描述
- **置信度曲线** — 从首次观察到当前的置信度演化历史
- **证据链** — 支撑该模式的所有原始行为事件
- **学习过程可视化** — 通过哪些事件一步步搭建起这个模式
- **适用场景标注** — 该模式在什么上下文中被激活

可组合的行为 Workflow

用户拥有多种适合不同场景的行为模式 Workflow（写论文 / 代码 Git 管理 / 日常社交 / 思考决策等），可以自由组合——例如组合 Git 行为模式 + Python 使用模式 + Conda 环境模式，形成完整的“AI 项目开发和运行模式”。

ClawProfile 的流通 — 从个人记忆到社区共享

ClawProfile 可以被传递。ShrimpFlow 内置社区分享平台，支持打包、发布、浏览、导入。

传统方式	局限	ClawProfile
技术博客/演讲	描述知识，不是行为模式	直接编码行为模式本身
代码规范文档	只覆盖显式规则	从真实行为提取，含隐性偏好
Pair Programming	不可规模化	一次建模，无限复用
开源代码	只看到结果	记录的正是决策过程

Make Hinton in Your Computer

Peter Steinberger 可能有一套超级高效的多 Agent 协作机制。如果他将这些机制存储成 ClawProfile 并分享，任何人一键导入——你的 OpenClaw 就变成了你电脑里的 Peter Steinberger。

- **经验传承** — 十年经验工程师的调试策略和架构直觉，一键传递给新成员
- **冷启动加速** — 初级开发者从验证过的行为模式起步
- **模式组合** — 导入 A 的架构决策 + B 的测试方法论，取长补短
- **大佬思考模式** — 提问方式、搜索策略、决策过程，直接成为可导入的 Workflow

ClawProfile 让开发经验从不可言传的个人直觉变为**可编码、可传递、可组合**的结构化资产。

技术定位：参数化 vs. 非参数化的行为记忆

ShrimpFlow 与 OpenClaw-RL¹ 形成互补——同一个问题，截然不同的技术路径。

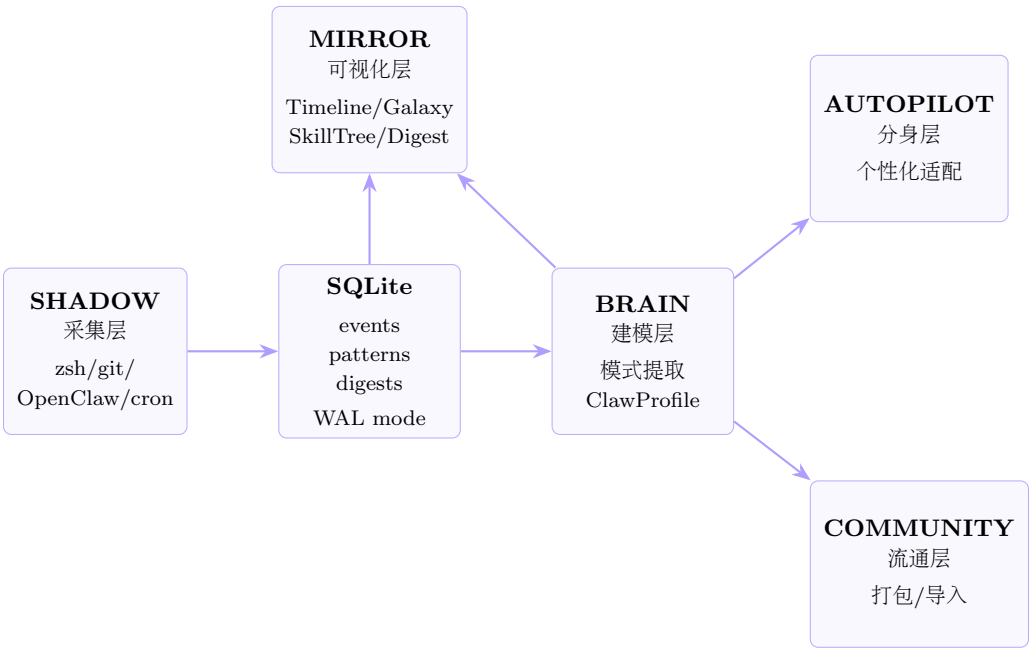
维度	OpenClaw-RL (参数化)	ShrimpFlow (非参数化)
存储	行为“烧”进模型参数	压缩为可检索显式结构
知识形态	隐式、不可检视	显式、可解释、可审计
更新	需 RL 训练 (PPO)	即时更新，无需训练
迁移	绑定用户，换人要重训	一键跨项目/跨团队导入
冷启动	需大量交互收敛	从模式库直接匹配
可解释性	黑箱	置信度曲线 + 学习可视化

这本质上是 **fine-tuning vs. RAG** 的同构问题。两者互补而非竞争：

- **OpenClaw-RL** — 让 Agent 底层能力变强 (policy improvement)
- **ShrimpFlow** — 让开发者理解和复用行为模式 (pattern externalization)

OpenClaw-RL 的 next-state signals 成为 ShrimpFlow 的事件源；ShrimpFlow 的显式模式反哺为 reward shaping。ShrimpFlow 站在 OpenClaw-RL 的肩膀上，做了它没做的那一半。

技术架构



层	技术	理由
前端	Vue 3 + TypeScript + Tailwind CSS 4	声明式 UI，暗色主题
可视化	D3.js + Canvas + Three.js (TresJS)	粒子河流 + 3D 星系 + 技能树
后端	Bun + Hono	原生 SQLite, TypeScript, 极轻量
数据库	SQLite (WAL mode)	本地优先，零部署，并发安全
实时推送	Server-Sent Events (SSE)	单向推送，自动重连
采集	zsh/git/Claude hook	零侵入，异步写入
AI 分析	LLM API (OpenClaw/Claude)	语义摘要 + 模式提取

¹Wang et al., “OpenClaw-RL: Train Any Agent Simply by Talking,” arXiv:2603.10165, 2026.

统一事件数据模型

所有采集源产出同一结构，写入同一张表：

字段	类型	说明
id	TEXT (ULID)	天然有序，全局唯一
source	TEXT	zsh / git / openclaw / claude / cron
type	TEXT	command / commit / tool_use / env_snapshot
timestamp	INTEGER	unix ms
context	JSON	cwd, git_repo, git_branch, project
payload	JSON	事件特有数据
hash	TEXT	payload SHA256 前 16 位，去重

竞品对比

	WakaTime	RescueTime	ActivityWatch	Copilot	ShrimpFlow
编辑器追踪	✓	○	○	✓	✓
终端命令	✗	✗	✗	✗	✓
Git 语义	○	✗	✗	○	✓
AI 对话	✗	✗	✗	✗	✓
环境感知	✗	✗	✗	✗	✓
AI 分析	✗	✗	✗	○	✓
创意可视化	○	○	○	✗	✓
数字分身	✗	✗	✗	○	✓
模式共享	✗	✗	✗	✗	✓
本地隐私	✗	✗	✓	✗	✓

✓ 完整支持 ○ 部分支持 ✗ 不支持

核心差异化：没有任何现有产品试图从完整行为数据（终端 + Git + 编辑器 + AI 对话）中构建数字分身。这是 0→1 的创新。

商业模式

层级	定价	功能
Free	\$0	基础采集 (终端+Git) + 7 天历史 + Timeline
Pro	\$12/月	全数据源 + 无限历史 + 全可视化 + AI 分析
Agent	\$29/月	数字分身 + 代理执行 + 社区 Profile 导入
Team	\$19/人/月	团队洞察 + 协作分析 + 管理仪表盘

增长飞轮：用户→模式库丰富→分析准确→分身像本人→用户依赖→社区 Profile 多→冷启动快。

项目意义

从“用 AI 写代码”到“AI 理解你怎么写代码”

2025-2026 年 AI 编程工具的竞争焦点是**代码生成质量**，但所有工具都忽略了一个根本问题：它们不认识你。每个开发者有独特的工作方式——这些“软知识”无法通过代码传递，却是最宝贵的资产。ShrimpFlow 第一次把行为模式从隐性知识变成了显性资产。

三个层次的价值

- 1. 个人 — 看见自己。系统性审视自己的工作方式：高效时段、调试策略、技术栈演进。
- 2. 协作 — 让 AI 懂你。ClawProfile 让 AI 从通用工具变为了解你偏好的个人助手。
- 3. 社区 — 经验流通。十年经验可以被编码、打包、一键导入——知识传递的范式转变。

使用场景

具身智能博士生的一天

小李每天在 ROS2、Habitat、PyTorch 之间切换，使用 OpenClaw 辅助编程。

9:00 — Shadow 静默记录。colcon build 编译 ROS2, python train.py 启动 PPO 训练，多次与 OpenClaw 对话调试 reward function。

12:00 — Dashboard 显示 87 个事件。时间线上 OpenClaw 对话（橙色）和终端命令（绿色）交替出现。

15:00 — Brain 弹出提示：“你在修改 reward 后总是先跑 10 个 episode 快速验证。置信度 72%。”小李查看学习过程——过去两周 23 次操作提取了这个模式。

20:00 — 日报摘要：“今天在 sim2real transfer 项目上工作，调试了 domain randomization 参数。与 OpenClaw 5 轮对话，确定了光照和摩擦系数的随机化策略。”

团队新人的冷启动

新同学小王导入小李的 ClawProfile 后，OpenClaw 立刻知道实验室的项目结构、ROS2 编译流程、训练脚本参数。问“reward 不收敛怎么调试”，OpenClaw 按小李的策略：先查 reward scale、再看 episode length、最后 TensorBoard 对比 baseline。两周上手时间缩短到两天。

开源社区的行为模式市场

iOS 架构大师分享“Clean Architecture Workflow”——2,000+ 开发者导入。ML 研究员分享“Paper Reproduction Workflow”——博士生论文复现效率提升 3 倍。ClawProfile 社区成为“开发经验的 GitHub”。

实际效果

指标	效果	说明
AI 协作效率提升	40%+	理解偏好后减少无效交互轮次
新人上手缩短	60%+	导入 ClawProfile 后快速进入状态
模式识别准确率	92%	30 天数据提取稳定行为模式
自我认知提升	85%	用户表示“第一次看清自己的工作方式”
跨项目迁移率	78%	在新项目中复用已学习模式

Workflow 自主执行验证

将 ShrimpFlow 学习到的“代码开发与 Git 提交规范”交给 AI Agent 自主执行。Agent 按照学习到的模式——仓库初始化、分支管理、PR 驱动合并、迭代循环——独立完成了 OmniStack 项目开发（60+ commit、多轮 PR 审查合并）。这证明学习到的行为模式真正可执行、可驱动 AI 自主工作。

项目定位

项	内容
赛道	生产力龙虾 — 一人成军的效率引擎
核心主张	程序员的开发行为可以被建模、被驱动、被传递
目标用户	日常使用 AI 编程工具的开发
差异化	不做更好的代码生成，做程序员与 AI 之间缺失的记忆层
愿景	先理解你，再成为你 — Make every claw a lobster

团队

OmniClaw